

PODS

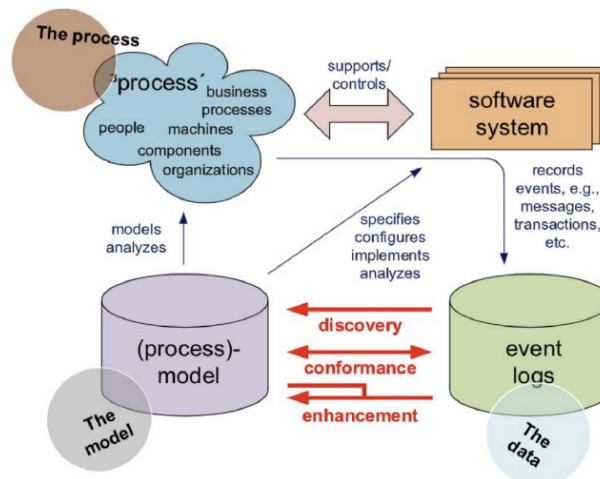
Davide Volpi

October 27, 2025

Contents

1	Context	2
1.1	Software System	2
1.2	Event Logs	2
1.3	Process Model	2
2	Petri Nets	3
2.1	Formalization	3
2.2	Preset and Postset	3
2.3	Enabledness	3
2.4	Firing Rule	3
2.5	Process	3
2.6	Process Models vs Petri Nets	4
2.7	State Space	4
2.8	Soundness	4
2.8.1	Formal Definition	5
2.8.2	Checking Soundness	5
2.9	Petri Nets Properties	5
2.9.1	Reachability	5
2.9.2	Other Properties	5
2.10	Special Classes of Petri Nets	6
2.10.1	Free-Choice Petri Nets	6
2.10.2	Block-Structured Petri Nets	6
3	Process Discovery	7
3.1	Workflow Log	7
3.2	Ordering Relations	7
4	Alpha-Family Discovery Algorithms	8
4.1	α -Algorithm	8
4.1.1	Log Completeness	9
4.1.2	Implicit Places	9
4.1.3	Structured Workflow-Nets	9
4.1.4	Rediscovering	9
4.2	$\alpha + -$ Algorithm	9
4.3	$\alpha + +- $ Algorithm	10

1 Context



1.1 Software System

Business Process: collection of activities that take one or more kinds of input and create an output that is of value to the customer.

Process-Oriented Information System (POIS): generic software system that is driven by explicit process representations to coordinate the enactment of business processes.

Process-Orchestration: a system acts as a central agent that controls the execution of the process activities, very similar to a conductor centrally controlling the musicians in an orchestra.

Integration logic is encoded in process model, where a workflow engine executes the integration process.

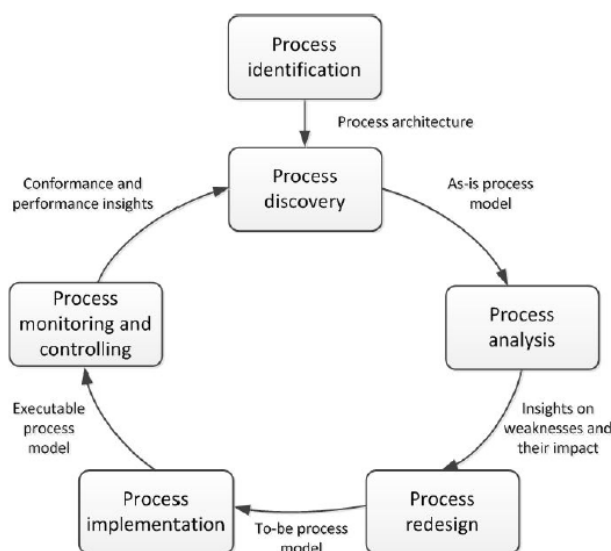
Human Interactions Workflows include user interaction process execution, combination of manual and fully automated activities, Active control of process by interaction with process participants.

1.2 Event Logs

Event: happening of interest, having a timestamp and carrying data.

Event Type: type for events of similar structure and semantics.

1.3 Process Model



Mapping: activities, building blocks that describe elementary pieces of work, routing conditions, describe temporal and logical constraints on the execution of activities, inputs, outputs, informational or physical artefacts processed by activities, events (how much time, messages, exceptions that influence the execution), resources are all mapped to a process model.

Prospectives: control flow, organisational structures, data structures, IT landscapes.

2 Petri Nets

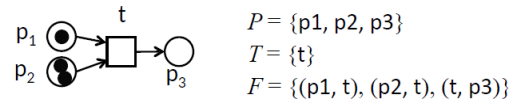
Any discrete dynamic system can be described as a transition system. Petri nets provide a compact way to describe (potentially infinite) transition systems as a directed graph consisting of places and transitions, where:

- **Transitions** are represented as **boxes** (unlabelled transitions can exist).
- **Places** are represented as **circles** and can contain ≥ 0 tokens.
- The **states of a transition** system are defined as the **distribution of tokens** over places.
- The Firing Rule govern the state changes.

2.1 Formalization

A Petri Net is a triple (P, T, F) where:

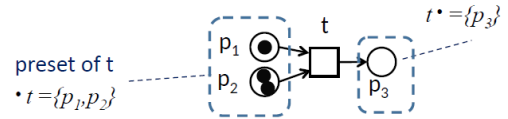
- P is a finite set of places.
- T is a finite set of transitions.
- $F \subseteq (P \times T) \cup (T \times P)$ is a flow relation.



2.2 Preset and Postset

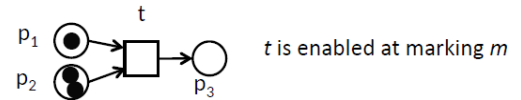
Define input and output places in terms of the flow relation. Let t be a transition:

- Set $= \{p \mid (p, t) \in F\}$ defines all input places of t (the preset of t).
- Set $= \{p \mid (t, p) \in F\}$ defines all output places of t (the postset of t).



2.3 Enabledness

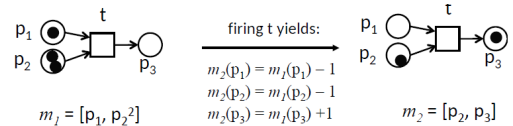
In a Petri Net (P, T, F) , a transition $t \in T$ is enabled at marking $m : P \rightarrow \mathbb{N}$ if for all $p \in \bullet t$, $m(p) > 0$.



2.4 Firing Rule

For a Petri Net (P, T, F) , a firing transition $t \in T$ at a marking m_1 leads to a new marking m_2 such that for any $p \in P$:

- $m_2(p) = m_1(p) - 1$ if $p \in \bullet t$
- $m_2(p) = m_1(p) + 1$ if $p \in t \bullet$
- $m_2(p) = m_1(p)$ otherwise



2.5 Process

Any time ≥ 2 activities are performed in the context of a case, fundamental principles of processes apply. Activities and processes are related. They have a logical order in which they have to be executed.

Sound Workflow Model: all the token have a path that leads to the final state.

The aim of a process is going to an initial place to a output place.

- **Proper Begin:** the only place without input arcs is the initial place.
- **Proper End:** the only place without output arcs is the output place.

2.6 Process Models vs Petri Nets

Process

- Activity by resource
- Decision by environment
- Process Model
- Case
- State of a case

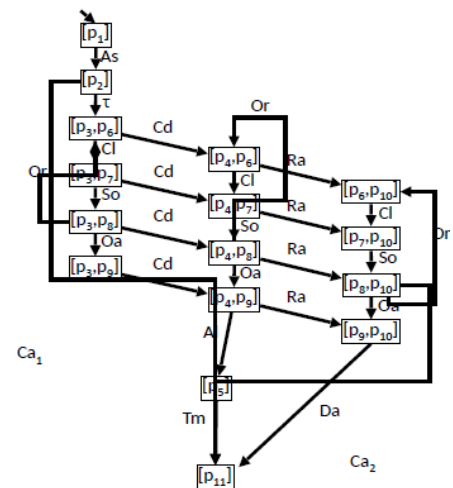
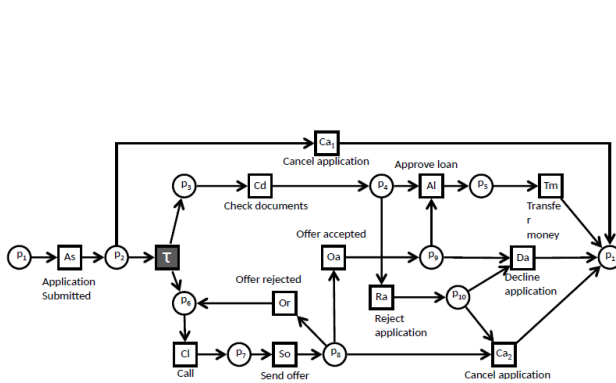
Petri Net

- Transition
- Transition
- Petri net explaining the flow of each case
- Token(s) in the net
- Marking of the Petri Net

When Petri Nets are used to model systems, the marking denotes the state of the system.
When Petri Nets are used to model processes, the marking denotes the state of an individual case.

2.7 State Space

1. We start from the initial marking.
2. Fire transitions and note down all new markings.
3. Do the same for all markings until all have been investigated.



2.8 Soundness

Workflow nets (or process models) are Petri Nets:

- with a **single source place i** .
- with a **single final place f** .
- where **every node is on a path from i to f** .

A **process model is sound** iif:

- **For every marking reachable from $[i]$, you can reach the marking $[f]$.**
- **No marking m can be reached with $m > [f]$.**
- **There are no dead transitions (for every transition t , a marking m can be reached from $[i]$ such that m enables t).**

Unsound models may get stuck, may never be able to terminate, cannot be simulated/analyzed and cause problems in the processes they model.

A workflow model is sound, iif the short-circuited net with marking $[p_1]$ is live and bounded.

2.8.1 Formal Definition

A workflow system (PN, i) with a workflow net $PN = (P, T, F)$ is sound if and only if:

- For every state M reachable from state i there exists a firing sequence from M to o , i.e.;

$$\forall M(i \xrightarrow{*} M) \implies (M \xrightarrow{*} o)$$

- State o is the only state reachable from state i with at least one token in place o , i.e. :

$$\forall M(i \xrightarrow{*} M \wedge M \geq o) \implies M = o$$

- There are no dead transitions in the workflow net in state i , i.e. :

$$(\forall t \in T) \exists M, M' : i \xrightarrow{*} M \xrightarrow{*} M'$$

2.8.2 Checking Soundness

Check soundness based on the reachability graph (RG):

1. Check whether from each node in the RG, there is a path to state o .
2. Check whether place o is marked only in state o .
3. Check whether each transition occurs in the RG.

2.9 Petri Nets Properties

2.9.1 Reachability

The reachability graph is the reachable portion of the transition system:

1. Label the initial marking m_0 as the root and tag it "new".
2. While "new" markings exists, do:
 - (a) Select a new marking m .
 - (b) If no transitions are enabled at m , tag m "dead end".
 - (c) While exist enabled transitions at m , do the following for each enabled transition t at m :
 - i. Obtain the marking m' that results from firing t at m .
 - ii. If m' does not appear in the graph add m' and tag it "new".
 - iii. Draw an arc with label t from m to m' (if not already present).
3. Output the graph.

Is important to state that:

- Given a Petri Net in a marking m_1 , is it possible to execute a sequence of transitions to reach marking m_2 ? If so, then m_2 is reachable from m_1 .
- Given a Petri Net in a marking m_1 . Is it possible to execute a sequence of transitions to reach some marking m_2 with a specific property p ?

2.9.2 Other Properties

- **Boundedness:** is the number of tokens in each place p less than k in all reachable marking m_2 (for all m_2 and p holds $m_2(p) \leq k$).
- **Dead transition:** is it possible to reach a marking m_2 in which transition t is enabled (or $m_2 \geq \bullet t$)? If so, t is not dead.
- **Deadlock:** is it possible to reach a marking in which no transition is enabled (or for all transitions $t, m_2 < \bullet t$).
- **Reversibility:** if the initial marking is m_1 , does it hold that for all reachable markings m_2 , we can always reach m_1 again?
- **Liveness:** for every reachable marking m_1 and transition t , is it possible to reach a marking m_2 in which t is enabled ($m_2 \geq \bullet t$).

Live (L), Deadlock Free (DF), Bounded (B), Reversible (R)

	Live and Deadlock Free (L,DF)	Live and not Deadlock Free (L,NDF)	Not Live and Deadlock Free (NL,DF)	Not Live and Not Deadlock Free (NL,NDF)
Bounded and Reversible (B,R)				
Bounded and Not Reversible (B,NR)				
Not Bounded and Reversible (NB,R)				
Not Bounded and Not Reversible (NB,NR)				

2.10 Special Classes of Petri Nets

2.10.1 Free-Choice Petri Nets

A Petri Net (P, T, F) is free-choice net iff $\forall t_1, t_2 \in T$ either $\bullet t_1 = \bullet t_2$ or $\bullet t_1 \cap \bullet t_2 = \emptyset$.

Non-free-choice nets should be avoided as much as possible in Business Process Modelling because the behaviour depends on the order in which transitions enabled at the same time fire. For most of non-free-choice nets, there is a free-choice net that preserves the same behaviour.

The properties are:

- Form the basis of languages like BPMN.
- Liveness and boundedness can be decided in polynomial time in the size of the model.
- Free choice nets are easy to understand as the behavior is local (all decisions are made locally by transitions sharing input places).

2.10.2 Block-Structured Petri Nets

The basis block of a block-structured Petri Net is a trivial workflow net with a single transition.

A block-structured Petri Net is composed of a hierarchy of block-structured Petri Nets by merging input and output places.

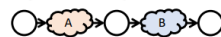
• Basis:

• Composition rules:

given:

and:

• Sequence composition:



• Parallel composition:



• Choice composition:



• Loop composition

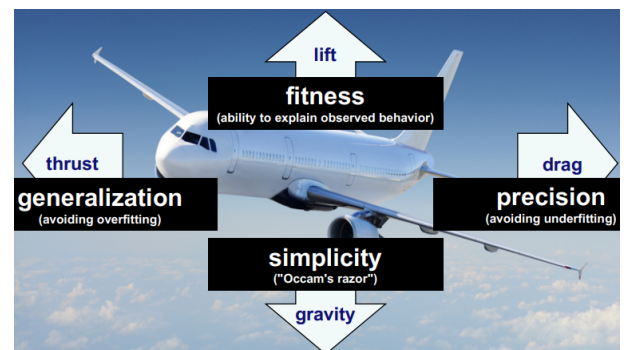
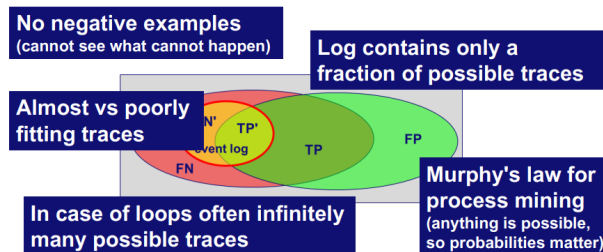


3 Process Discovery

We want to create a process model for observed behaviour. Typically not every possible behaviour may have been executed and thus recorded. The quality dimensions are:



- **Fitness:** model should allow for behaviour seen in log.
- **Precision-Generalisation trade-off:** model does not allow for behaviour completely unrelated to log, but generalizes to some extent what is seen in the log.
- **Simplicity:** discovered model should be as simple as possible.



3.1 Workflow Log

General notion of event log:

- Format of log entries: (timestamp, caseID, activity, additional attributes ...)
- Various abstractions can be applied.

One abstraction: the notion of a workflow log

- Set of all distinct sequences of activity executions.
- Let T be a set of activities (tasks).
- Let T^* the set of all finite sequences over T .
- $\sigma \in T^*$ is a trace, all tasks in σ belong to the same case.
- $W \subseteq T^*$ is a workflow log.

Assumption: each task occurs at most once in a process model.

3.2 Ordering Relations

Log-based ordering relations for a pair of tasks $a, b \in T$ in a workflow log W :

- **Direct successor:** $a >_w b$ iff b directly follows a in at least one trace.
- **Casuality (symmetrical):** $a \rightarrow_w b$ iff $a >_w b$ and not $b >_w a$.
- **Concurrency (symmetrical):** $a \parallel_w b$ iff $a >_w b$ and $b >_w a$.
- **Exclusiveness:** $a \#_w b$ iff not $a >_w b$ and not $b >_w a$.

4 Alpha-Family Discovery Algorithms

Simple algorithm: α -algorithm

- Detect current execution of activities.
- Relatively easy, certain properties can be proven.
- Not robust against noise, incomplete or erroneous logs.
- Therefore, nice for illustration, but limited use in practice.

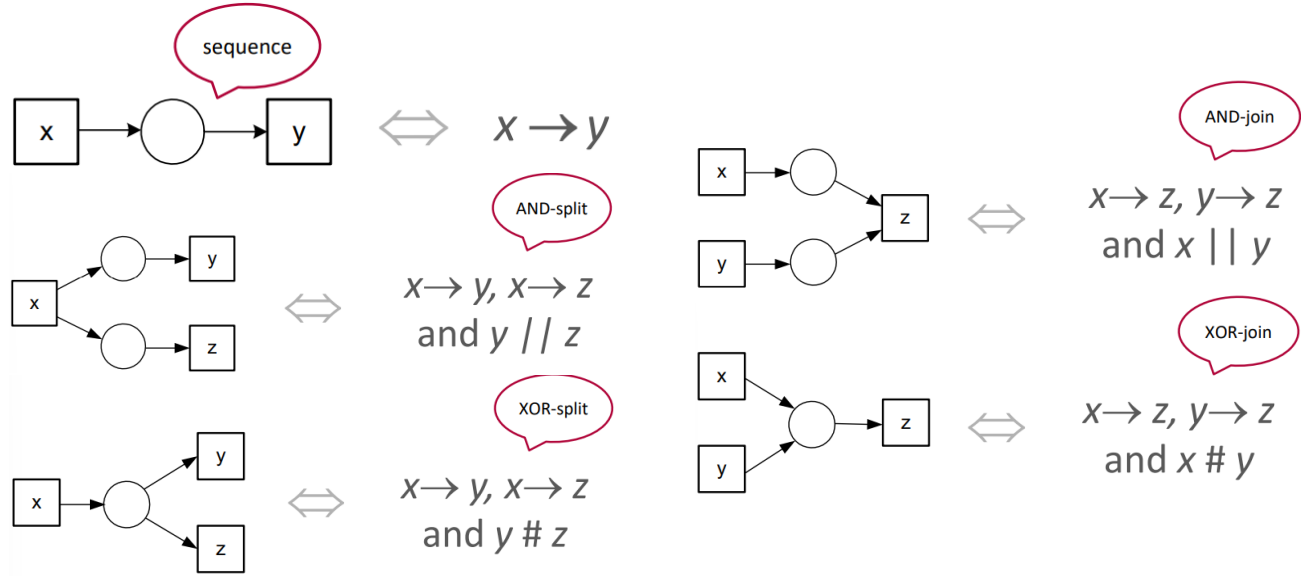
Extensions: $\alpha^{(+)}$ -algorithms

- α^+ and α^{++} extend a α -algorithm to broaden the spectrum of constructs that may be discovered.
- Still not robust against noise.

4.1 α -Algorithm

Idea: Create workflow net based on the ordering relations, such that the ordering is obeyed by the net.

Realisation: derive a Petri Net fragment from the each entry of the ordering relations.



Let W be a workflow log over T . $\alpha(W)$ is defined as follows:

1. $T_W = \{t \in T \mid \exists \sigma \in W, t \in \sigma\}$ all the traces
2. $T_I = \{t \in T \mid \exists \sigma \in W, t = \text{first}(\sigma)\}$ all initial places of the traces
3. $T_O = \{t \in T \mid \exists \sigma \in W, t = \text{last}(\sigma)\}$ all final places of the traces
4. $X_W = \{(A, B) \mid A \subseteq T_W \wedge B \subseteq T_W \wedge \forall a \in A \forall b \in B, a \rightarrow_W b \wedge \forall a_1, a_2 \in A, a_1 \#_W a_2 \wedge \forall b_1, b_2 \in B, b_1 \#_W b_2\}$ all the $\rightarrow$ relations as tuples
5. $Y_W = \{(A, B) \in X_W \mid \forall (A', B') \in X_W, A \subseteq A' \wedge B \subseteq B' \implies (A, B) = (A', B')\}$ merge the $\rightarrow$ tuples into triplets where possible and simplify (#)
6. $P_W = \{p_{(A, B)} \mid (A, B) \in Y_W\} \cup \{i_W, o_W\}$ create the petri net with the previously created transitions and add i/o places
7. $F_W = \{(a, p_{(A, B)}) \mid (A, B) \in Y_W \wedge a \in A\} \cup \{(p_{(A, B)}, b) \mid (A, B) \in Y_W \wedge b \in B\} \cup \{(i_W, t) \mid t \in T_I\} \cup \{(t, o_W) \mid t \in T_O\}$
8. $\alpha(W) = (P_W, T_W, F_W)$

4.1.1 Log Completeness

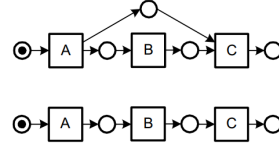
Required completeness for the α -algorithm:

- Completeness wrt direct successorship ($>_W$).
- Whenever two tasks may occur as direct successors, this must be observed in at least one trace.

This completeness criterion reduced the number of required traces for highly concurrent processes dramatically.

4.1.2 Implicit Places

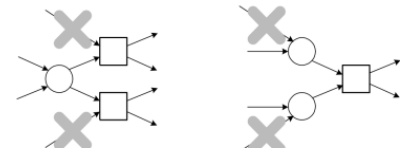
The presence or absence of implicit places does not change the behaviour of a net system. Hence, process models with implicit places cannot be rediscovered. This is not really an issue, since there are no consequences for the behaviour.



4.1.3 Structured Workflow-Nets

A WF-Net $N = (P, T, F)$ is an SWF-Net (Structured Workflow Net) iff:

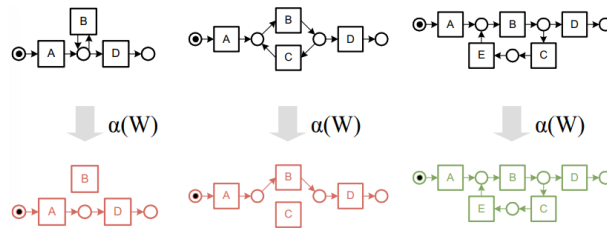
1. For all $p \in P$ and $t \in T$ with $(p, t) \in F : |\bullet p| > 1$ implies $|\bullet t| = 1$.
2. For all $p \in P$ and $t \in T$ with $(p, t) \in F : |\bullet t| > 1$ implies $|\bullet p| = 1$.
3. There are no implicit places.



Sufficiently expressive to model most of process-related control-flow structures, sequences, concurrency, exclusive choices, ...

4.1.4 Rediscovering

Short Loops (length 1/2) pose an issue for rediscoverability by the α -algorithm. The idea is to extend



α -algorithm $\rightarrow \alpha + -$ algorithm, so that sound SWF-Nets with short loops can be rediscovered.

4.2 $\alpha + -$ Algorithm

Redefine ordering relations:

- **Director successor** $a >_w b$ and exclusiveness $a \#_w b$ as before.
- **Causality** $a \rightarrow_w b$ iff $a >_w b$ and not $b >_w a$ or there exists a trace containing aba .
- **Concurrency** $a \parallel_w b$ iff $a >_w b$ and $b >_w a$ and there exists no trace containing aba

Loops of length 2 and concurrency can be distinguished, length 2 and 1 cannot be distinguished.

Solution to cope with loops of length 1:

- Pre-processing: identify transitions in loop of length 1 by sequence $\dots aa \dots$
- Remove all these transitions from all observed sequences.
- Mining: proceed as before with redefined ordering relations and the assumption of loop completeness of the log.
- Post-processing: insert loops of length 1 into constructed net.
- Find place p to insert loop for transition a
 - Find transitions, which are direct predecessors of a , but not direct successors of a .
 - Find transitions, which are direct successors of a , but not direct predecessors of a .
 - Input/output places of these transitions indicate where the loop has to be inserted.

$\alpha + -$ algorithm fails to rediscover indirect dependencies imposed by non-free choice constructs.

4.3 $\alpha + + -$ Algorithm

Additional behavioural relations:

- XOR-split relation: $a < | b$ iff $a \# b$ and there exists a c such that $c \rightarrow a$ and $c \rightarrow b$.
- XOR-join relation: $a | > b$ iff $a \# b$ and there exists a c such that $a \rightarrow c$ and $b \rightarrow c$.
- Indirect successor: $a >> b$ iff not $a > b$ and there exists a reachable firing sequence in which a happens before b , such that there is no XOR-split or -join between a and b .

The model is not robust to noise.